

DHBW Food App

Technische Dokumentation

Projektmanagement · 2. Semester · DHBW Karlsruhe
14. June 2026

Inhaltsverzeichnis

1	Projektübersicht	2
1.1	Funktionsumfang	2
2	Systemarchitektur	3
2.1	Überblick	3
2.2	Container-Setup	3
2.3	CORS und Proxy	3
3	Backend	4
3.1	Technologie	4
3.2	REST-API-Endpunkte	4
3.3	Datenbankschema	4
4	Frontend	5
4.1	Technologie und Aufbau	5
4.2	Design-System	5
4.3	Datums-Navigation	5
4.4	XSS-Schutz	5
5	Lokale Entwicklung	6
5.1	Voraussetzungen	6
5.2	Setup	6
5.3	Wichtige Befehle	6
5.4	Hinweise	6

1 Projektübersicht

Die **DHBW Food App** ist eine webbasierte Anwendung für Studierende der DHBW Karlsruhe. Sie ermöglicht das Einsehen des tagesaktuellen Mensaplans, das Bewerten von Gerichten sowie das Einreichen von Essensvorschlägen. Ziel ist es, Studierenden eine einfache Möglichkeit zu bieten, Feedback zur Mensa zu geben und so zur Qualitätsverbesserung beizutragen.

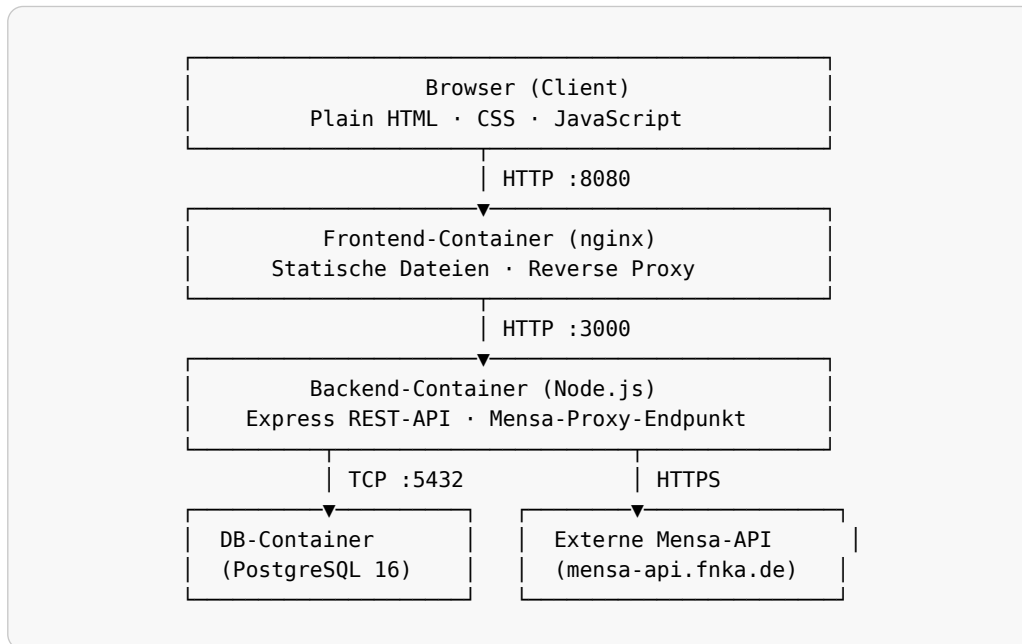
1.1 Funktionsumfang

Funktion	Beschreibung
Tagesmenü	Live-Abruf des aktuellen Mensaplans über eine externe API; Anzeige nach Kantine gefiltert
Datums-Navigation	Navigation zwischen Werktagen; vergangene Tage bewertbar, zukünftige Tage nur zur Vorschau
Bewertung	Sternbewertung (1–5) für Gesamteindruck, Geschmack und Portionsgröße inkl. optionalem Kommentar
Rankings	Anzeige der Top-, Flop- und Trend-Gerichte basierend auf gespeicherten Bewertungen
Statistiken	Gesamtanzahl Bewertungen, Durchschnittswert, Bewertungsverteilung als Balkendiagramm
Vorschläge	Formular zum Einreichen neuer Gerichtsideen; Übersicht aller bisherigen Vorschläge

2 Systemarchitektur

2.1 Überblick

Die Anwendung folgt einer klassischen **3-Tier-Architektur** und wird vollständig in Containern betrieben.



2.2 Container-Setup

Alle drei Container werden über eine einzige `docker-compose.yml` definiert und über ein internes Bridge-Netzwerk (`dhbw_food_network`) verbunden. Der Datenbankcontainer ist mit einem Health-Check ausgestattet; das Backend startet erst, wenn die Datenbank bereit ist (`depends_on: condition: service_healthy`).

Container	Image	Port	Aufgabe
<code>dhbw_food_frontend</code>	<code>nginx:alpine</code>	<code>8080:80</code>	Ausliefern der statischen Dateien
<code>dhbw_food_backend</code>	<code>node:alpine</code>	<code>3000:3000</code>	REST-API & Mensa-Proxy
<code>dhbw_food_db</code>	<code>postgres:16-alpine</code>	intern	Datenpersistenz

Der Datenbankinhalt wird in einem benannten Docker-Volume (`dhbw_food_db_data`) gespeichert und überlebt Container-Neustarts.

2.3 CORS und Proxy

Die externe Mensa-API erlaubt keine direkten Browser-Anfragen (kein CORS-Header für `localhost`). Das Backend fungiert daher als **transparenter Proxy**: Der Endpoint `GET /api/mensa-plan/:date` leitet die Anfrage serverseitig weiter und gibt das Ergebnis an den Browser zurück. Dadurch wird das CORS-Problem vollständig umgangen.

3 Backend

3.1 Technologie

Das Backend basiert auf **Node.js** mit dem **Express**-Framework. Die Datenbankanbindung erfolgt über den offiziellen **pg**-Treiber mit Connection Pooling (**pg.Pool**).

3.2 REST-API-Endpunkte

Methode	Endpunkt	Beschreibung
GET	/health	Health-Check; gibt { status: "ok" } zurück
GET	/api/mensa-plan/:date	Proxy zur externen Mensa-API für das angegebene Datum (Format: YYYY-MM-DD)
POST	/api/mensa-dishes/sync	Upsert von Mensa-Gerichten in die DB; gibt { name → id }-Mapping zurück
GET	/api/dishes	Alle heute verfügbaren Gerichte inkl. Durchschnittsbewertung
POST	/api/ratings	Neue Bewertung speichern (dish_id, gesamt sind Pflichtfelder)
GET	/api/rankings?filter=	Top / Flop / Trend-Ranking (mind. 1 Bewertung erforderlich)
GET	/api/stats	Gesamtstatistiken inkl. Bewertungsverteilung
GET	/api/proposals	Alle eingereichten Vorschläge, absteigend nach Datum
POST	/api/proposals	Neuen Vorschlag einreichen (name, kategorie sind Pflichtfelder)

3.3 Datenbankschema

Die PostgreSQL-Datenbank besteht aus drei Tabellen:

- **gerichte** – Mensagerichte mit Name, Kategorie, Preis und Verfügbarkeitsdatum. Ein Unique-Index auf (name, verfuegbar_am) verhindert Duplikate beim täglichen Sync.
- **bewertungen** – Nutzerbewertungen mit Fremdschlüssel auf **gerichte**. Enthält Gesamt-, Geschmack- und Portionswertung (je 1–5) sowie einen optionalen Kommentar.
- **vorschlaege** – Frei einreichbare Gerichtsvorschläge mit Name, Kategorie und Begründung.

Das Schema wird beim ersten Start des Datenbankcontainers über **init.sql** automatisch angelegt (idempotent mit **IF NOT EXISTS**). Der Unique-Index auf **gerichte** wird zusätzlich beim Backend-Start idempotent migriert.

4 Frontend

4.1 Technologie und Aufbau

Das Frontend ist eine **Single-Page Application (SPA)** ohne JavaScript-Framework. Es basiert auf Plain HTML, CSS und JavaScript nach dem **IIFE-Modulpattern** (Immediately Invoked Function Expression). Jede Seite der App ist als eigenes Modul gekapselt, das seine öffentliche Schnittstelle über `window.*Module` exponiert.

Datei	Aufgabe
<code>utils.js</code>	Geteilte Hilfsfunktionen: <code>escHtml()</code> (XSS-Schutz), <code>KATEGORIE_EMOJI</code> -Mapping
<code>api.js</code>	Alle HTTP-Aufrufe ans Backend (<code>mensaApi</code> , <code>api</code>); Mensa-Daten-Parsing und DB-Sync
<code>bewertung.js</code>	Tagesmenü-Ansicht, Datums-Navigation, Kantinenfilter, Bewertungs-Modal
<code>ranking.js</code>	Rankings-Ansicht, Statistik-Karten, Bewertungsverteilung als CSS-Balkendiagramm
<code>vorschlaege.js</code>	Vorschlagsformular mit clientseitiger Validierung, Vorschlags-Übersicht
<code>app.js</code>	Seitennavigation (Hash-Routing), Lazy-Loading der Module

4.2 Design-System

Das UI folgt einem einheitlichen Design-System mit **DHBW-Rot (#E2001A)** als Primärfarbe. Alle Abstände, Radien, Schatten und Farben sind als CSS Custom Properties in `:root` definiert. Komponenten-Styles sind in `components.css` ausgelagert, globale Basis-Styles in `style.css`.

4.3 Datums-Navigation

Die Bewertungsseite enthält einen Werktags-Navigator (Vor/Zurück-Pfeile). Wochenenden werden beim Navigieren automatisch übersprungen. Gerichte vergangener Tage und des aktuellen Tages sind bewertbar. Für zukünftige Tage (bis 5 Werktage im Voraus) werden die Gerichte nur zur Vorschau angezeigt – der Bewerten-Button ist ausgeblendet und die Karten sind nicht klickbar.

4.4 XSS-Schutz

Alle dynamisch eingefügten Nutzer- oder API-Daten werden über `escHtml()` escaped, bevor sie als HTML gerendert werden. Die Funktion nutzt die native DOM-API (`createTextNode`) und ist damit sicherer als reguläre Ausdrücke.

5 Lokale Entwicklung

5.1 Voraussetzungen

- Podman oder Docker mit Compose-Plugin
- Ports 8080 und 3000 müssen frei sein

5.2 Setup

```
# Repository klonen und in das Code-Verzeichnis wechseln
cd DHBW-Food-WebApp/Code
```

```
# Container bauen und starten
podman-compose up --build
# oder
docker compose up --build
```

Die App ist anschließend unter `http://localhost:8080` erreichbar. Das Backend antwortet unter `http://localhost:3000`.

5.3 Wichtige Befehle

Befehl	Beschreibung
<code>podman-compose up --build</code>	Container (neu) bauen und starten
<code>podman-compose down</code>	Container stoppen und entfernen
<code>podman ps</code>	Laufende Container anzeigen
<code>podman logs dhw_food_backend</code>	Backend-Logs einsehen
<code>podman logs dhw_food_db</code>	Datenbanklog einsehen

5.4 Hinweise

- **Wochenenden:** Die Mensa ist samstags und sonntags geschlossen. Die App zeigt an diesen Tagen eine entsprechende Meldung statt Gerichte zu laden.
- **Mensa-API:** Die externe API (`mensa-api.fnka.de`) ist nur an Werktagen verfügbar. Bei Ausfall wird der Fehler im Frontend angezeigt.
- **Datenbankpersistenz:** Das Volume `dhw_food_db_data` bleibt auch nach `podman-compose down` erhalten. Für einen vollständigen Reset: `podman volume rm dhw_food_db_data`.